Moduł 11 – Wydajność – Zadania Teoria SQL

Przygotowanie - korzystając ze skryptu poniżej utwórz obiekty potrzebne do zadania.

=== > LINK DO SKRYPTU < ===

W skryptach do sprawdzenia, nie podawaj proszę wartości wynikowych tylko samą składnię z ewentualnym opisem.

1. Przygotuj zapytanie wyświetlające dane sprzedażowe za okres ostatnich 2 miesięcy (skorzystaj ze składni INTERVAL). W wyniku wyświetl wszystkie atrybuty sprzedażowe i dodatkowo nazwę i kod produktu oraz region, w którym produkt powstał.
   Dane wyświetl wyłącznie dla kodu produktu równego PRD8.

2. Korzystając z opcji **EXPLAIN ANALYZE,** przeanalizuj plan zapytania dla zapytania z zadania 1. Rozpisz, z jakich elementów się składa: rodzaj użytego algorytmu, koszty na poszczególnych etapach, jaki rodzaj pobierania danych został wykorzystany (sekwencyjne skanowanie czy indeksy).

3. Oblicz miarę selektywności dla atrybutu PRODUCT_CODE z tabeli PRODUCTS.

4. Dodaj indeks do tabeli PRODUCTS na polu PRODUCT_CODE typu BTREE.

5. Zweryfikuj plan wykonania zapytania dla zapytania z zadania 1 po dodaniu indeksu. Czy indeks został użyty?

6. Dodaj indeks dla daty sprzedaży (SAL_DATE) na tabeli SALES.

7. Zweryfikuj plan wykonania zapytania dla zapytania z zadania 1 po dodaniu indeksu. Czy indeks dla SAL_DATE lub PRODUCT_CODE został użyty?

8. Na podstawie instrukcji poniżej zweryfikuj czy partycjonowanie tabeli ma istotny wpływ na plan wykonania zapytania i operację INSERT. (ten skrypt znajduje się również w linku powyżej).

```
DROP TABLE sales, sales_partitioned CASCADE;

CREATE TABLE sales (
      id SERIAL,
      sal_description TEXT,
      sal_date DATE,
      sal_value NUMERIC(10,2),
      sal_prd_id INTEGER,
      added_by TEXT DEFAULT 'admin',
      created_date TIMESTAMP DEFAULT now()
);
```

```
CREATE TABLE sales_partitioned (
      id SERIAL,
      sal_description TEXT,
      sal_date DATE,
      sal_value NUMERIC(10,2),
      sal_prd_id INTEGER,
      added_by TEXT DEFAULT 'admin',
      created_date TIMESTAMP DEFAULT now()
) PARTITION BY RANGE (sal_date);

CREATE TABLE sales_y2018 PARTITION OF sales_partitioned
    FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');

CREATE TABLE sales_y2019 PARTITION OF sales_partitioned
    FOR VALUES FROM ('2019-01-01') TO ('2020-01-01');

CREATE TABLE sales_y2020 PARTITION OF sales_partitioned
    FOR VALUES FROM ('2020-01-01') TO ('2021-01-01');

EXPLAIN ANALYZE
INSERT INTO sales (sal_description, sal_date, sal_value,
sal_prd_id)
     SELECT left(md5(i::text), 15),
            CAST((NOW() - (random() * (interval '60 days')))
AS DATE),
            random() * 100 + 1,
            floor(random() * 10)+1::int
       FROM generate_series(1, 1000000) s(i);

EXPLAIN ANALYZE
INSERT INTO sales_partitioned (sal_description, sal_date,
sal_value, sal_prd_id)
     SELECT left(md5(i::text), 15),
            CAST((NOW() - (random() * (interval '60 days')))
AS DATE),
            random() * 100 + 1,
            floor(random() * 10)+1::int
 FROM generate_series(1, 1000000) s(i);
```